ACICP302ICS | MODULE 1 PRACTICAL LAB REPORT

Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation

Source-Code-Guided OT Simulation Lab: safe preparation, loopback verification and normal-operation baseline

Learner	Abubakar Adamu
Registration number	C11/26/ACIS/17353
Module	ACICP302ICS: Integrated ICS Network Security & Attacks
Assessment	Module 1 Practical Lab (100 marks, 50% practical pass threshold)
Institution	International Cybersecurity and Digital Forensics Academy (ICDFA)
Lab session	13 October 2026 (VM system clock, 17:57 to 18:50)
Environment	Ubuntu 64-bit guest in VMware Workstation, loopback (127.0.0.1) only
Course source commit	`2a66f9bcf253eb3fe31c68ad52794cd535421da8`

1. Executive summary

This report documents my work on the ICDFA Module 1 practical lab, which uses a course-supplied, loopback-only Modbus/TCP simulation of a small hydrogen ("H2 Facility") process. The lab is source-code-guided: the simulation is supplied, and the analysis, captures and conclusions are my own.

What has been completed and evidenced

- Prepared an Ubuntu VM in VMware Workstation using a NAT adapter (not Bridged), installed build tools, Git and Python dependencies, and created a Python virtual environment for the lab (Figures 1, 2, 7).
- Cloned the course repository and recorded the exact commit, `2a66f9bcf253eb3fe31c68ad52794cd535421da8`, working from `labs/ot-security/student-lab-source/` (Figures 3 to 5).
- Verified by source search that all 13 host/target settings in `plant_sim/` are `127.0.0.1`, and that neither the archived address `192.168.2.139` nor a `0.0.0.0` wildcard bind appears in the supplied source (Figure 5, Table 4).
- Created the evidence workspace `~/evidence/` with `screenshots`, `logs`, `pcaps` and `reports` folders (Figure 5).
- Observed the unguarded ("UNDEFENDED, NO GUARD ACTIVE") dashboard at `http://127.0.0.1:8080` in a normal state: 64.0 °C, 38.2 bar and 88.6 Nm³/h, all inside the displayed safe bands (Figure 8, Table 6).

Main observations so far

- Tank pressure sits at about 82% of its displayed 30 to 40 bar band, the least headroom of the three process variables, so it is the variable most worth watching in later drift and false-data scenarios.
- The direct dashboard path has no guard in front of it by design, which makes it the natural baseline for comparing guarded and unguarded behaviour in Activity 06.
- The `docs/reference-architecture.md` file named in the brief was not found inside `student-lab-source/` (Figure 6), so architecture claims in this draft rely on the directory listing, the brief and the README until the document is located.

Still to be evidenced: packet capture (Activity 03), exposure mapping (04), guard analysis (05), at least three approved scenarios with recovery proof (06), and cleanup and defence material (07/08). Section 5 lists exactly what to capture for each.

2. Source, scope and safety (Activity 00)

2.1 Source record

Item	Recorded value
Course repository	https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks
Canonical source path	`/home/sirdiq/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source` (confirmed with `pwd`, Figure 5)
Commit (`git rev-parse HEAD`)	`2a66f9bcf253eb3fe31c68ad52794cd535421da8` (Figures 4 and 5)
Clone result	130 objects received, 80.87 KiB, 24 deltas resolved (Figures 3 and 4)
Top level of student source	`README.md`, `plant_sim/`, `requirements.txt` (Figure 5)
Archived original	`archive-original/` was not executed. Every command in the evidence was run from `student-lab-source/`.

Table 1. Source and version record.

2.2 Environment and network isolation

Item	Recorded value
Hypervisor / guest	VMware Workstation, guest "Ubuntu 64-bit". Package versions in Figure 2 (for example `build-essential 12.12ubuntu2.26.04.2`) point to Ubuntu 26.04.
Resources	4 GB memory, 2 processors, 30 GB disk (Figure 1)
Network adapter	**NAT**. Bridged is not selected (Figure 1).
Adapter state in Figure 1	Device status shows "Connected" and "Connect at power on" ticked. This is the setup-phase state.
Guest user / host name	`sirdiq@sirdiq-VMware-Virtual-Platform`
Python / packages	Python 3.14 virtual environment (`.venv`); `pymodbus 3.6.9`, `Flask 3.1.3`, plus Werkzeug 3.1.9, Jinja2 3.1.6, Click 8.5.0, itsdangerous 2.2.0, blinker 1.9.0, MarkupSafe 3.0.4 (Figure 7)

Table 2. VM and isolation record.

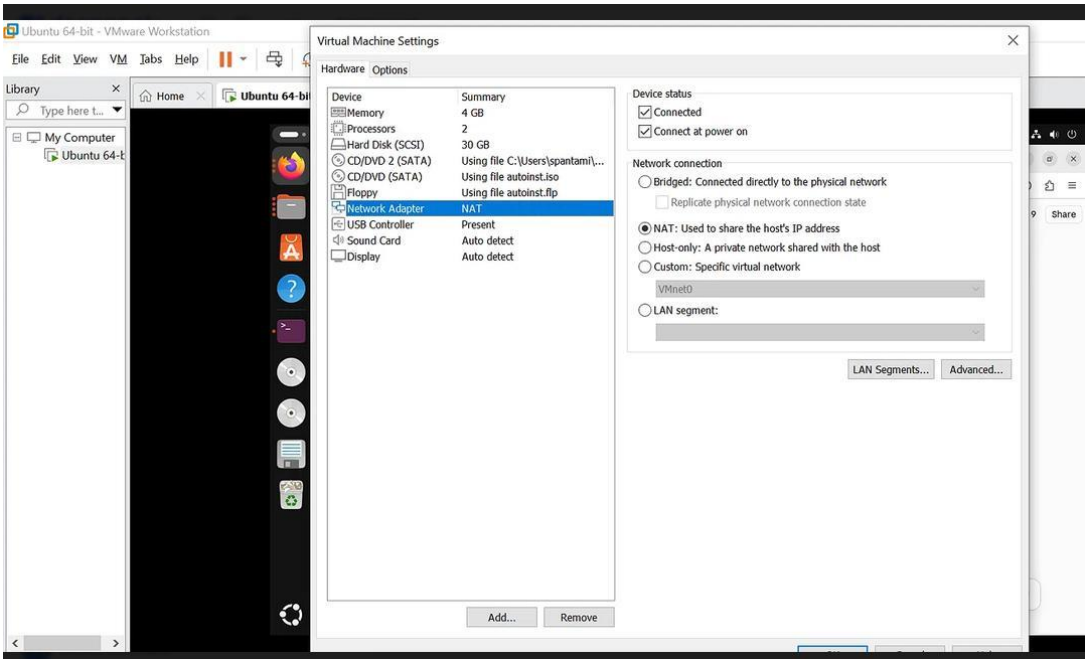


Figure 1. *VMware Virtual Machine Settings: 4 GB RAM, 2 CPUs, 30 GB disk, Network Adapter set to NAT (not Bridged).*

2.3 Setup timeline

Time (13 Oct)	Action	Evidence
17:57	Installed build tools, Git and Python development packages with `apt` (`build-essential`, `git`, `python3-dev`, `python3-pip`, `g++`).	Figure 2
18:01	Created `~/course` and cloned the course repository.	Figure 3
18:05	Clone command repeated with identical object counts; commit hash read with `git rev-parse HEAD`.	Figure 4
18:15	Commit re-read; loopback search of `plant_sim`; evidence folders created.	Figure 5
18:30	Listed `plant_sim/`; looked for `docs/reference-architecture.md` (not found).	Figure 6
18:34	Installed `requirements.txt` into the virtual environment.	Figure 7
18:50	Observed the direct dashboard on `127.0.0.1:8080`.	Figure 8

Table 3. Chronology reconstructed from the on-screen clock in each screenshot.

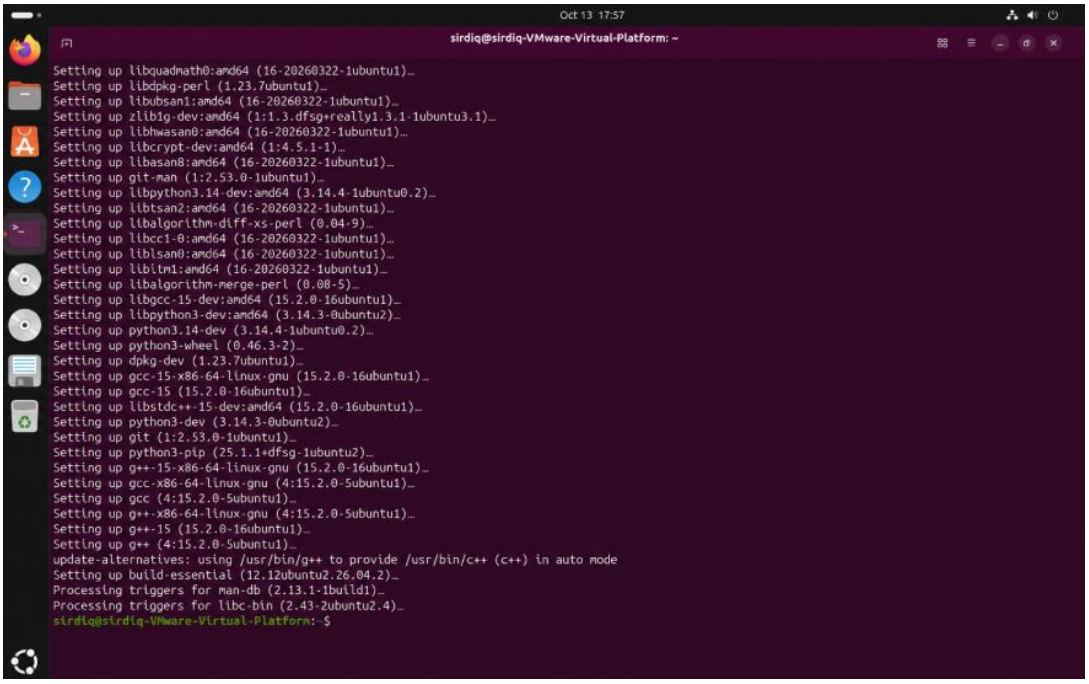


Figure 2. *Terminal at 17:57: apt finishing installation of build-essential, g++, gcc-15, git and python3-dev.*

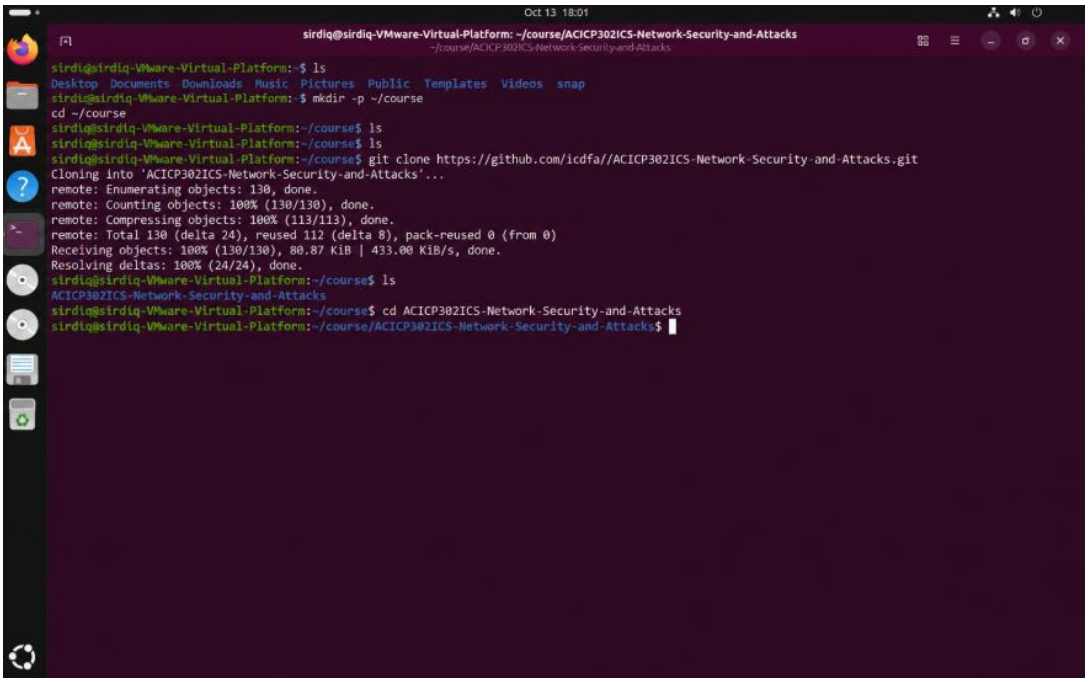


Figure 3. *Terminal at 18:01: creating ~/course and cloning the course repository (130 objects, 80.87 KiB).*

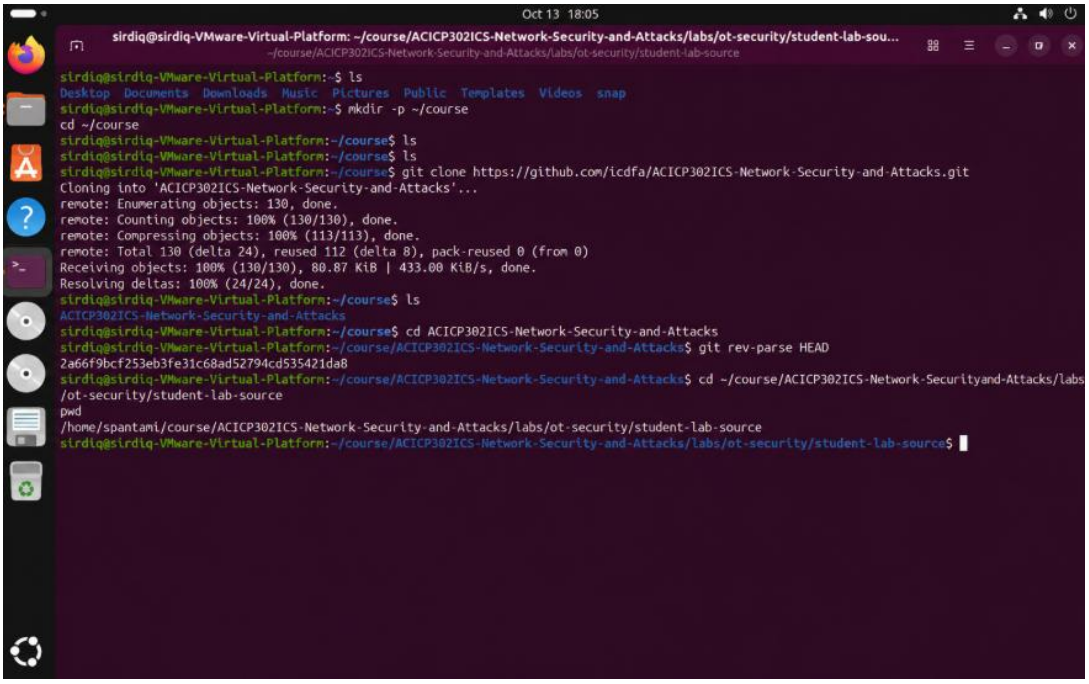


Figure 4. *Terminal at 18:05: repository cloned again, commit read with git rev-parse HEAD (2a66f9bc…1da8), then changing into labs/ot-security/student-lab-source.*

2.4 Loopback verification

The brief requires every client target and service bind to be `127.0.0.1`, and requires a stop if that check fails. I searched `plant_sim/` for the archived destination `192.168.2.139` and the wildcard bind `0.0.0.0`: **no matches were returned**. I then listed every `127.0.0.1` occurrence:

File	Line	Setting	Value	Role (as interpreted)
`plant.py`	20	`HOST`	127.0.0.1	Plant / Modbus service address
`guard.py`	19	`PLANT_HOST`	127.0.0.1	Guard upstream (plant) target
`guard.py`	21	`HOST`	127.0.0.1	Guard service address
`dashboard.py`	6	`TARGET_HOST`	127.0.0.1	Direct dashboard data source
`dashboard.py`	224	`app.run(host, port=8080)`	127.0.0.1:8080	Direct dashboard web bind
`dashboard_guarded.py`	8	`TARGET_HOST`	127.0.0.1	Guarded dashboard data source
`dashboard_guarded.py`	265	`app.run(host, port=8081)`	127.0.0.1:8081	Guarded dashboard web bind
`recon_scan.py`	5	`HOST`	127.0.0.1	Client script target
`replay_attack.py`	5	`HOST`	127.0.0.1	Client script target
`false_data_attack.py`	5	`HOST`	127.0.0.1	Client script target
`slow_drift_attack.py`	5	`HOST`	127.0.0.1	Client script target
`attack_hold.py`	5	`HOST`	127.0.0.1	Client script target
`dos_attack.py`	6	`HOST`	127.0.0.1	Client script target

Table 4. Output of the loopback search (13 matches in 10 files). The "Role" column is my interpretation from variable names and must be confirmed against the source when it is read in Activity 01.

Result: the loopback check passes and no stop condition was triggered. The Modbus/TCP port number is not visible in the evidence captured so far and is recorded as **unknown** until read from `plant.py`.

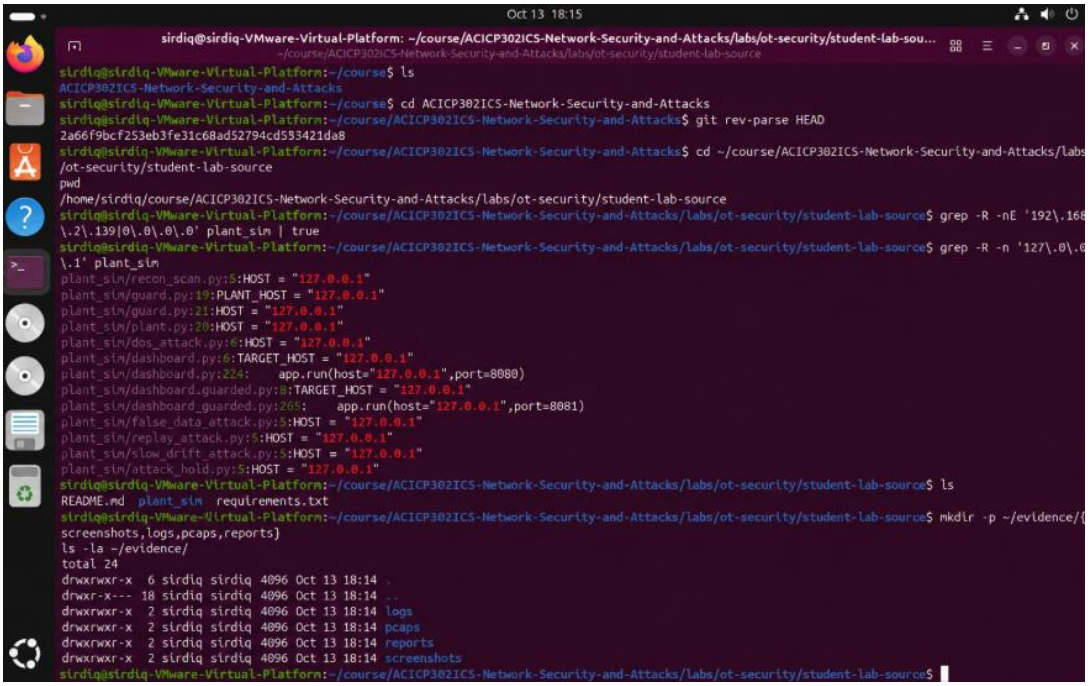


Figure 5. *Terminal at 18:15: commit hash, working directory, loopback search results across plant_sim, and creation of ~/evidence/{screenshots,logs,pcaps,reports}.*

2.5 Evidence workspace

The folder `~/evidence/` was created at 18:14 with four sub-folders: `screenshots`, `logs`, `pcaps` and `reports` (Figure 5). This workspace will be committed to a private learner repository for submission.

2.6 Authorized scope, stop condition and recovery

Item	Statement
Authorized scope	Only the supplied `student-lab-source/` simulation, on `127.0.0.1`, inside the assigned Ubuntu VM. No external hosts, no Bridged networking, no execution of `archive-original/`.
Stop condition	Any target or bind that is not `127.0.0.1`, any real or external system, VM instability, unclear scope, or an unexpected result: stop, notify the instructor, follow the recovery plan.
Recovery method	Stop all services and scripts, confirm the lab ports are no longer listening, restore the assigned clean snapshot if directed.

3. Architecture and plant baseline (Activity 01)

3.1 Supplied components

`ls -la plant_sim/` (Figure 6) shows ten Python files. Roles below come from the brief and the file names; they are provisional until each file is read.

File	Size (B)	Provisional role
`plant.py`	3,270	Simulated process and Modbus/TCP service
`dashboard.py`	11,683	Direct dashboard, served on port 8080 (line 224, Table 4)

File	Size (B)	Provisional role
`guard.py`	9,198	Supplied defensive guard (the brief names range, step and drift checks)
`dashboard_guarded.py`	13,161	Guarded dashboard, served on port 8081 (line 265)
`recon_scan.py`	465	Supplied client script (name suggests discovery)
`replay_attack.py`	550	Supplied demonstration script (name suggests replay)
`false_data_attack.py`	2,115	Supplied demonstration script (name suggests false data)
`slow_drift_attack.py`	619	Supplied demonstration script (name suggests gradual drift)
`attack_hold.py`	1,771	Supplied demonstration script (name suggests holding a value)
`dos_attack.py`	616	Availability/load demonstration. **Needs explicit instructor approval before it is run.**

Table 5. Contents of `student-lab-source/plant_sim/`, plus `README.md` and `requirements.txt` at the top level.

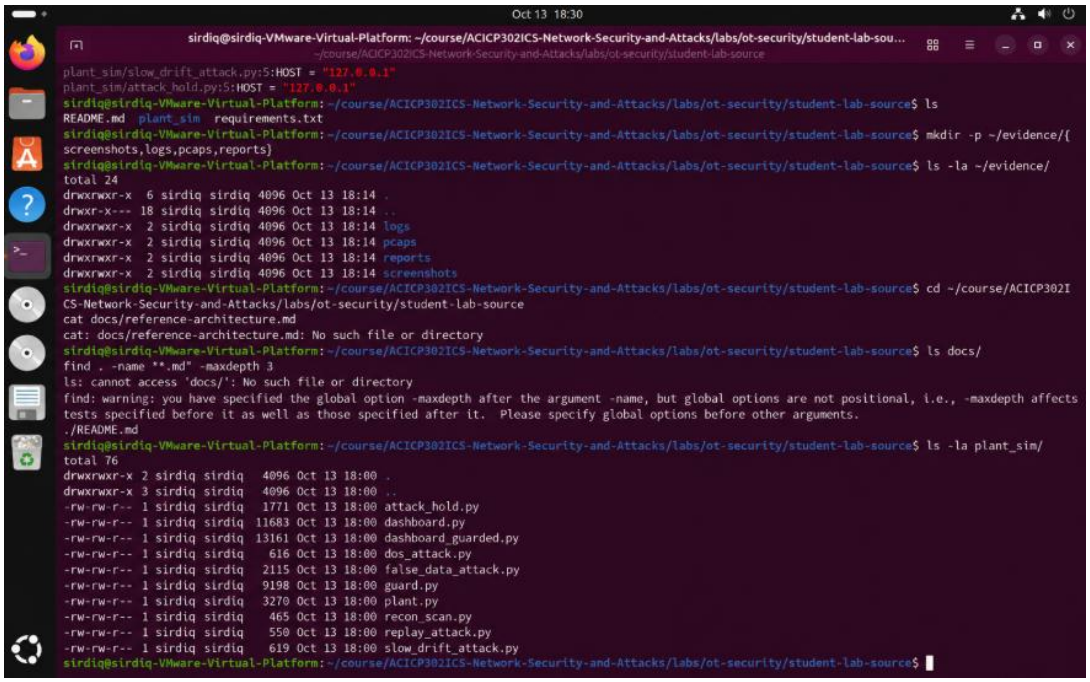


Figure 6. *Terminal at 18:30: listing of plant_sim/ and the failed lookup of docs/reference-architecture.md (no such file or directory under student-lab-source).*

> **Gap to close:** `docs/reference-architecture.md` is not under `student-lab-source/`. Check the repository root and the `labs/ot-security/` level (`find ~/course -name "reference-architecture.md"`), or ask the instructor, then read it before finalising the architecture diagram. Do not reconstruct its content from memory.

Dependencies were installed into a Python virtual environment before any service was started (Figure 7).

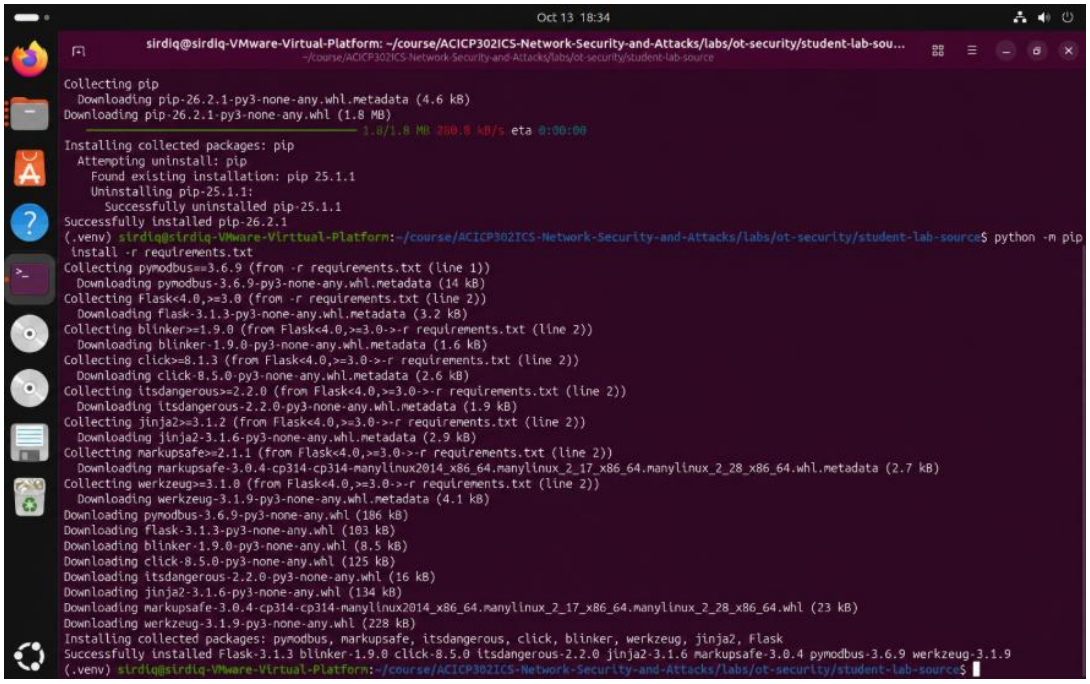


Figure 7. *Terminal at 18:34: pip install -r requirements.txt inside the virtual environment (pymodbus 3.6.9, Flask 3.1.3 and dependencies installed successfully).*

3.2 Data flow

Provisional flow, from the brief and file names: the plant simulation exposes process values over Modbus/TCP on loopback; the **direct dashboard** (port 8080) reads them without any guard; the **guard** sits between a client and the plant, and the **guarded dashboard** (port 8081) presents values via the guard. The dashboard banner confirms the direct path is unprotected: "UNDEFENDED, NO GUARD ACTIVE" (Figure 8).

3.3 Observed normal baseline

The direct dashboard was opened at `http://127.0.0.1:8080` at 18:50 (page footer: "Live, 18:50:20"). It reported **SYSTEM NORMAL** with the following values:

Process value	Observed	Displayed safe band	Position in band	Status
Electrolyzer temperature	64.0 °C	60 to 75 °C	27%	Inside band
Tank pressure	38.2 bar	30 to 40 bar	82%	Inside band, nearest upper limit (1.8 bar headroom)
H2 flow rate	88.6 Nm³/h	50 to 150 Nm³/h	39%	Inside band (labelled "Normal")
Compressor	RUNNING	not stated	n/a	Running
Export flow	88.6 Nm³/h	n/a	n/a	Equals H2 flow rate

Table 6. Baseline read from the direct dashboard. "Position in band" = (observed − min) ÷ (max − min), calculated by me from the displayed limits.

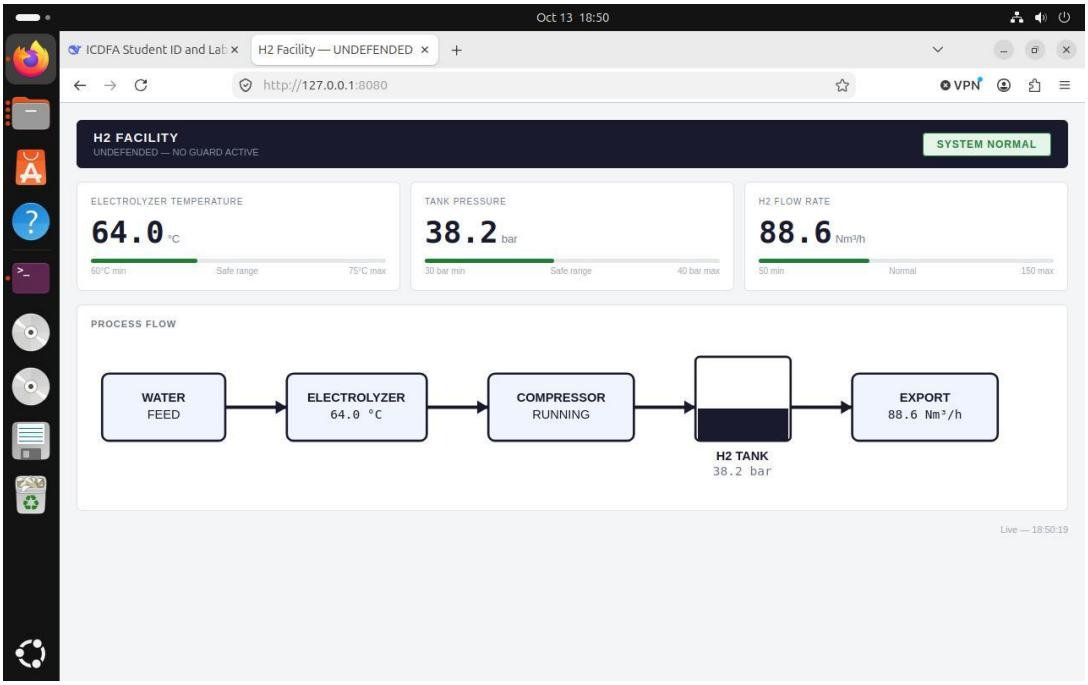


Figure 8. *Direct dashboard ("H2 Facility, UNDEFENDED, NO GUARD ACTIVE") at http://127.0.0.1:8080 showing SYSTEM NORMAL and the process-flow view: water feed, electrolyzer 64.0 °C, compressor RUNNING, H2 tank 38.2 bar, export 88.6 Nm³/h.*

Interpretation. The process-flow panel is internally consistent: the export rate matches the H2 flow tile, the compressor is running, and the tank pressure shown in the diagram matches the pressure tile. The bands printed under each tile are what the dashboard displays; whether they are identical to the thresholds the plant or guard actually enforce has not yet been checked against the source.

What this does not show. One snapshot does not demonstrate how values update over time, and no guarded-dashboard view has been captured, so differences between direct and guarded views are not yet known.

4. Modbus/TCP data model (Activity 02)

The brief requires a source-backed register map and says unknowns must be marked unknown rather than guessed. The table below lists what the dashboard evidence supports and leaves the register-level fields open.

Data point	Unit	Displayed band	Observed	Register address	Scale / type
Electrolyzer temperature	°C	60 to 75	64.0	UNKNOWN	UNKNOWN
Tank pressure	bar	30 to 40	38.2	UNKNOWN	UNKNOWN
H2 flow rate	Nm³/h	50 to 150	88.6	UNKNOWN	UNKNOWN
Compressor state	state	n/a	RUNNING	UNKNOWN	UNKNOWN
System status	text	n/a	SYSTEM NORMAL	UNKNOWN	UNKNOWN

Table 7. Data-point table. Values are shown with one decimal on the dashboard; whether the registers store scaled integers is unknown until `plant.py` is read.

5. Remaining evidence by activity

The following activities are not yet supported by screenshots. Each row says what to capture. Use supplied script defaults unchanged, run one scenario at a time, and obtain instructor approval for any availability/load scenario.

Activity	Capture and write up	Brief weighting
03 Packet analysis	Capture on the `lo` interface only, on the documented Modbus/TCP port, with normal services and an authorized read client running (no abnormal scenarios). Save a labelled `.pcapng` into `~/evidence/pcaps/`. In Wireshark, annotate one matched request and response: endpoints and port, transaction and protocol IDs, function code, address and value, result. Tie the value to Table 7 and the dashboard; state what the capture proves and does not prove.	Evidence item 3 (15 marks, shared with 04)
04 Exposure mapping	Scope card (VM, loopback target, method, time window, expected state, stop condition, recovery). Service inventory from the source and read-only checks, for example listening ports for 8080, 8081 and the Modbus port. Risk register entries with preventative and detective controls.	Evidence item 3
05 Guard analysis	Read `guard.py`. Build a decision table: check, input, condition, decision/action, related event or log evidence. Capture a normal pass and at least one event on the guarded dashboard (`8081`). Discuss limits: client authentication, direct-path access, availability, correlated values, threshold tuning, recovery.	Evidence item 4 (20 marks)
06 Controlled validation	Written test plan before running: script, path, baseline, expected result, evidence, stop condition, recovery. At least three approved scenarios, one at a time: script output, dashboard state, guard events. Expected-versus-observed table and recovery proof after each; report unexpected results honestly.	Evidence item 5 (15 marks)
07 / 08 Recovery and defence	Stop all services; show the lab ports are no longer listening; restore snapshot if directed. Final evidence log and index, risk register, comparison and recovery record, defence plan. Private repository URL with matching branch, commit and file references.	Evidence items 1 and 6

Table 8. Evidence still required.

6. Provisional risk register

Entries below are limited to what the current evidence supports. They will be extended and re-rated after Activities 04 to 06.

ID	Finding	Evidence	Potential consequence	Control
R1	Direct dashboard path has no guard in front of it.	Banner "UNDEFENDED, NO GUARD ACTIVE" (Figure 8)	Altered or false process values would reach the operator view unchecked (to be tested in Activity 06).	Preventative: use the guarded path. Detective: guard events and audit log.
R2	Tank pressure operates near its upper bound.	38.2 bar against a 30 to 40 bar band (Figure 8, Table 6)	Little headroom for gradual drift before the band is exceeded.	Rate-of-change and drift checks; review threshold margins.
R3	VM adapter was connected (NAT) during setup.	Figure 1	Unintended external reachability if left connected while the lab runs.	Disconnect the adapter before running services, as the brief requires.

Table 9. Provisional risk register.

7. Evidence index

ID	Description	Time (13 Oct)	Activity	Suggested repo path
E01	VMware network adapter settings (NAT)	not shown	00	`screenshots/E01-vm-network-settings.jpg`
E02	apt install of build tools and Git	17:57	00	`screenshots/E02-apt-setup.png`
E03	Repository clone	18:01	00	`screenshots/E03-git-clone.png`
E04	Clone repeated, commit hash	18:05	00	`screenshots/E04-commit-record.png`
E05	Commit, loopback search, evidence folders	18:15	00	`screenshots/E05-loopback-check.png`
E06	plant_sim listing; missing reference doc	18:30	00 / 01	`screenshots/E06-plant-sim-listing.png`
E07	Dependency installation in venv	18:34	00	`screenshots/E07-pip-install.png`
E08	Direct dashboard, normal state	18:50	01	`screenshots/E08-direct-dashboard.jpg`

Table 10. Index of the screenshots included in this report (Figures 1 to 8 correspond to E01 to E08 in the order they appear).